

```
$ cat communication_patterns.md
```

# API & Network Communication Patterns

A field guide to WebSockets, Webhooks, REST, GraphQL, SSE,  
Long Polling, Pub/Sub, and gRPC

---

|                                        |                                   |
|----------------------------------------|-----------------------------------|
| 01 REST .....                          | Request / Response over HTTP      |
| 02 GraphQL .....                       | Query language for APIs           |
| 03 WebSockets .....                    | Full-duplex persistent connection |
| 04 Server-Sent Events (SSE) .....      | One-way server push over HTTP     |
| 05 Long Polling .....                  | Held-open HTTP requests           |
| 06 Webhooks .....                      | Server-to-server callbacks        |
| 07 Pub/Sub .....                       | Decoupled event broadcasting      |
| 08 gRPC .....                          | High-performance RPC over HTTP/2  |
| 09 Comparison table & when to use what |                                   |

---

# 01

## // REpresentational State Transfer

REST is an architectural style for designing networked APIs around HTTP. Each request targets a **resource** (a URL, like /users/42) and uses a standard HTTP verb (GET, POST, PUT, PATCH, DELETE) to describe the action. Every request is independent — the server holds no memory of previous requests (this is called **statelessness**).

|             |                                                                       |
|-------------|-----------------------------------------------------------------------|
| Direction   | Client-initiated, request → response (one-shot)                       |
| Transport   | HTTP/1.1 or HTTP/2                                                    |
| Data format | Usually JSON, sometimes XML                                           |
| Best for    | CRUD operations, public APIs, resource-oriented backends              |
| Weak point  | Over-fetching / under-fetching data; many round trips for nested data |

### Typical flow

GET /api/users/42 → server looks up user 42 → returns 200 OK + JSON body. Connection closes after the response. Next request starts from scratch.

---

## 02

### // A query language for your API, not a transport

GraphQL flips REST's model: instead of many fixed endpoints, there's usually a single endpoint (/graphql) and the client sends a **query** describing exactly which fields it wants, across however many related resources. The server resolves that query against a schema and returns precisely that shape of data — nothing more, nothing less.

|             |                                                                                          |
|-------------|------------------------------------------------------------------------------------------|
| Direction   | Client-initiated, request → response (subscriptions add push)                            |
| Transport   | Usually HTTP POST; subscriptions often ride over WebSockets                              |
| Data format | JSON, shaped by the client's query                                                       |
| Best for    | Complex/nested data, mobile clients with limited bandwidth, aggregating multiple sources |
| Weak point  | Caching is harder than REST; N+1 query problems if resolvers aren't optimized            |

#### Typical query

```
query { user(id: 42) { name, posts { title } } } → one response containing exactly the user's name and their post titles, in a single round trip.
```

---

## 03

// One connection, full duplex, both sides can talk anytime

A WebSocket starts as a normal HTTP request that then **upgrades** to a persistent TCP connection. Once upgraded, both client and server can send messages to each other at any time, independently, without re-establishing a connection. This is **full-duplex** communication — unlike HTTP's strict request/response pattern.

|             |                                                                                     |
|-------------|-------------------------------------------------------------------------------------|
| Direction   | Bidirectional — either side can push, anytime                                       |
| Transport   | TCP, starts as HTTP then upgrades (ws:// or wss://)                                 |
| Data format | Text or binary frames, format is up to you (often JSON)                             |
| Best for    | Chat apps, live collaboration, multiplayer games, real-time dashboards              |
| Weak point  | Stateful connection = harder to scale horizontally; needs reconnect/heartbeat logic |

### Typical flow

Client: GET /ws (Upgrade: websocket) → Server: 101 Switching Protocols → connection stays open → either side sends frames whenever it wants, until either side closes it.

This is the pattern behind your superadmin WebSocket monitor log idea for DIETA-MATAHARI — the server can push new log lines the moment they happen, instead of the admin panel polling for them.

---

## 04

// One-way push from server to client, over plain HTTP

SSE is a simpler cousin of WebSockets: the client opens a normal HTTP connection and the server keeps it open, streaming a sequence of text events down to the client over time. It's **one-way only** (server → client) — the client can't send data back over the same connection, only make separate requests.

|             |                                                                               |
|-------------|-------------------------------------------------------------------------------|
| Direction   | One-way — server pushes to client only                                        |
| Transport   | Plain HTTP/1.1, kept open (Content-Type: text/event-stream)                   |
| Data format | UTF-8 text, simple event/data lines                                           |
| Best for    | Live feeds, notifications, progress updates, stock tickers                    |
| Weak point  | No client → server channel; limited browser connections per domain; text only |

### Typical flow

Client: GET /events (Accept: text/event-stream) → connection stays open → server streams: "data: {...}\n\n" repeatedly → browser's EventSource fires an event for each chunk.

---

## 05

### // Fake real-time, built on plain old HTTP requests

Long polling simulates push using only standard request/response. The client sends a request; instead of answering immediately, the server **holds the connection open** until it has new data (or a timeout hits), then responds. The client immediately re-opens a new request, and the cycle repeats. It predates WebSockets/SSE and still shows up as a fallback.

|             |                                                                                        |
|-------------|----------------------------------------------------------------------------------------|
| Direction   | Client-initiated, but response is delayed until data is ready                          |
| Transport   | Plain HTTP, one request held open at a time                                            |
| Data format | Usually JSON                                                                           |
| Best for    | Environments where WebSockets/SSE aren't available (old proxies, restrictive networks) |
| Weak point  | Higher latency and overhead than SSE/WebSockets; more connections churned              |

#### Typical flow

Client: GET /poll → server waits (up to ~30s) → new data arrives → server responds → client immediately sends GET /poll again.

---

## 06

// "Don't call us, we'll call you" – server-to-server callbacks

A webhook is just a plain HTTP POST that **a service you use sends to a URL you provide**, the moment something happens on their end. There's no persistent connection and no polling — you register a callback URL once, and the provider hits it whenever a relevant event occurs (a payment succeeds, a PR is opened, a build finishes).

|             |                                                                                       |
|-------------|---------------------------------------------------------------------------------------|
| Direction   | Provider-initiated, one-shot POST to your endpoint                                    |
| Transport   | Standard HTTP/HTTPS, no open connection between events                                |
| Data format | Usually JSON payload describing the event                                             |
| Best for    | Third-party integrations: payments, CI/CD, GitHub events, SaaS event notifications    |
| Weak point  | You must expose a public endpoint; need to verify signatures and handle retries/dupes |

### Typical flow

You register: `https://centaim.com/webhooks/github` → GitHub receives a push event → GitHub sends POST to that URL with the event payload → your server verifies it and reacts.

---

## 07

### // Publishers and subscribers, decoupled by a broker/topic

Publish/Subscribe is a messaging pattern rather than a specific protocol. **Publishers** emit messages to a **topic/channel** without knowing who's listening. **Subscribers** register interest in a topic and receive every message published to it. A message broker (Redis, Valkey, Kafka, RabbitMQ, NATS...) sits in the middle and handles fan-out delivery.

|             |                                                                                        |
|-------------|----------------------------------------------------------------------------------------|
| Direction   | Many-to-many, fully decoupled via a broker                                             |
| Transport   | Depends on broker — TCP protocols specific to Redis/Kafka/RabbitMQ/etc.                |
| Data format | Broker-dependent, often binary or JSON payloads                                        |
| Best for    | Event-driven microservices, fan-out notifications, decoupling producers from consumers |
| Weak point  | Added infra to run/maintain; message ordering & delivery guarantees vary by broker     |

#### Typical flow

Service A: PUBLISH orders.created {...} → broker fans it out → Service B and Service C, both SUBSCRIBED to orders.created, each receive their own copy independently.

Since Valkey is already in your DIETA-MATAHARI stack for caching, it can double as a lightweight pub/sub broker for exactly this kind of fan-out, without adding Kafka/RabbitMQ.



---

## 08

### // Strongly-typed RPC over HTTP/2, built for service-to-service calls

gRPC lets you call a method on a remote service as if it were a local function. You define services and message types in a **.proto** file (Protocol Buffers), and gRPC generates client/server code in your language of choice. It runs over HTTP/2, so it supports streaming in both directions on a single connection, and messages are compact binary rather than JSON.

|             |                                                                                          |
|-------------|------------------------------------------------------------------------------------------|
| Direction   | Unary, or streaming (client, server, or bidirectional)                                   |
| Transport   | HTTP/2, required                                                                         |
| Data format | Protocol Buffers (binary, schema-defined)                                                |
| Best for    | Internal microservice-to-microservice communication, low-latency/high-throughput needs   |
| Weak point  | Not natively browser-friendly (needs grpc-web/proxy); binary format isn't human-readable |

#### Typical flow

```
client stub calls GetUser(id: 42) → serialized to protobuf → sent over HTTP/2 → server
deserializes, runs handler, streams back a User message.
```

---

## 09

// side-by-side cheat sheet

| Pattern      | Direction                 | Connection                   | Best for                          |
|--------------|---------------------------|------------------------------|-----------------------------------|
| REST         | Client → Server           | One-shot per request         | Standard CRUD APIs                |
| GraphQL      | Client → Server           | One-shot (+ optional stream) | Flexible, nested data fetching    |
| WebSockets   | Bidirectional             | Persistent, full-duplex      | Real-time apps, chat, games       |
| SSE          | Server → Client           | Persistent, one-way          | Live feeds, notifications         |
| Long Polling | Client → Server (delayed) | Repeated held-open requests  | Push fallback, legacy support     |
| Webhooks     | Server → Server           | One-shot callback            | 3rd-party event integrations      |
| Pub/Sub      | Many → Many (via broker)  | Broker-managed               | Event-driven microservices        |
| gRPC         | Bidirectional (streams)   | Persistent, HTTP/2           | Internal service-to-service calls |

### Quick decision guide

- Need a public, resource-based API? → **REST**
- Clients need flexible, nested queries in one round trip? → **GraphQL**
- Need instant two-way updates (chat, live cursors, games)? → **WebSockets**
- Only need server → client push, over plain HTTP? → **SSE**
- Need push-like behavior but can't use WebSockets/SSE? → **Long Polling**
- A third-party service needs to notify your backend of events? → **Webhooks**
- Multiple services need to react to the same event, decoupled? → **Pub/Sub**
- Two internal services need fast, typed, high-throughput calls? → **gRPC**